第四届eBPF开发者大会

The 4th eBPF Developer Conference (eBPFDC 2026)

陕西 · 西安　　2026年4月18日

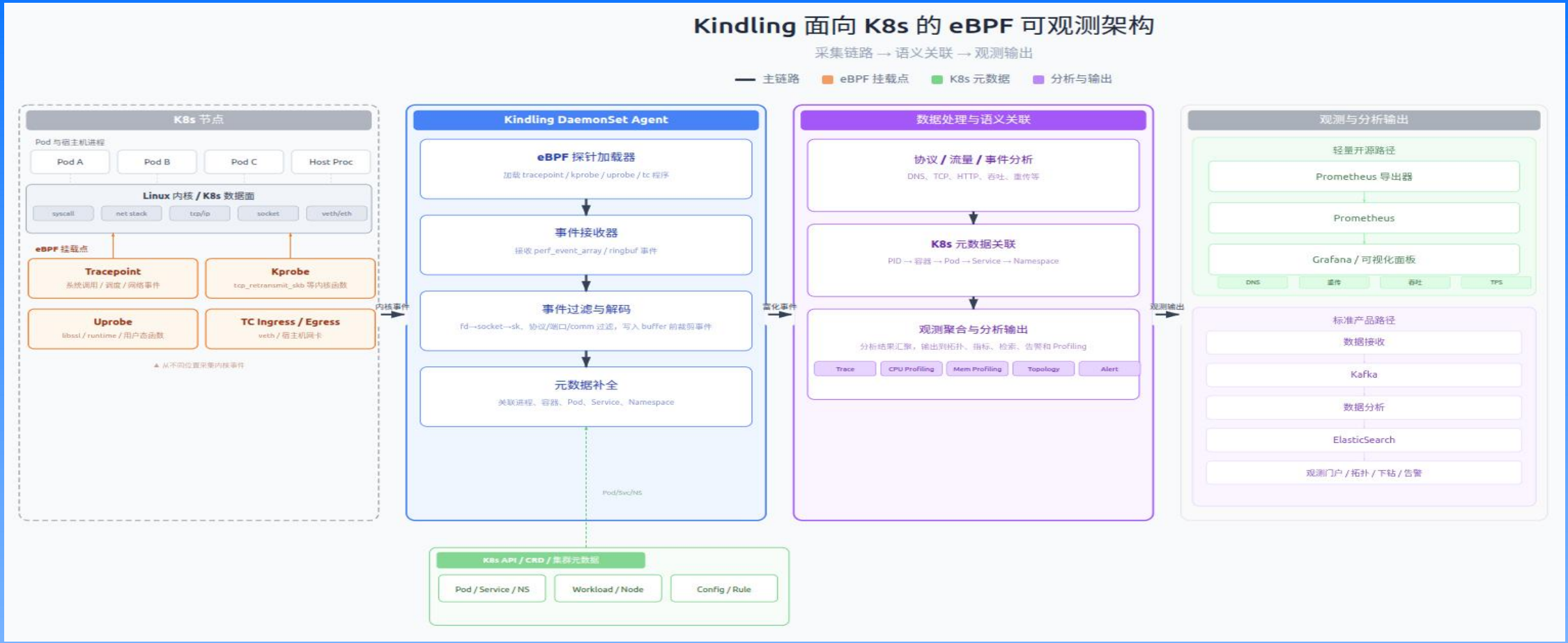
https://www.ebpftravel.com

面向 K8s 的 eBPF 可观测实践

Kindling
云原生流量转发
CPU / Memory Profiling
两个真实问题

陕西 · 西安 2026年4月18日

Kindling 整体架构：从节点采集到 K8s 语义观测

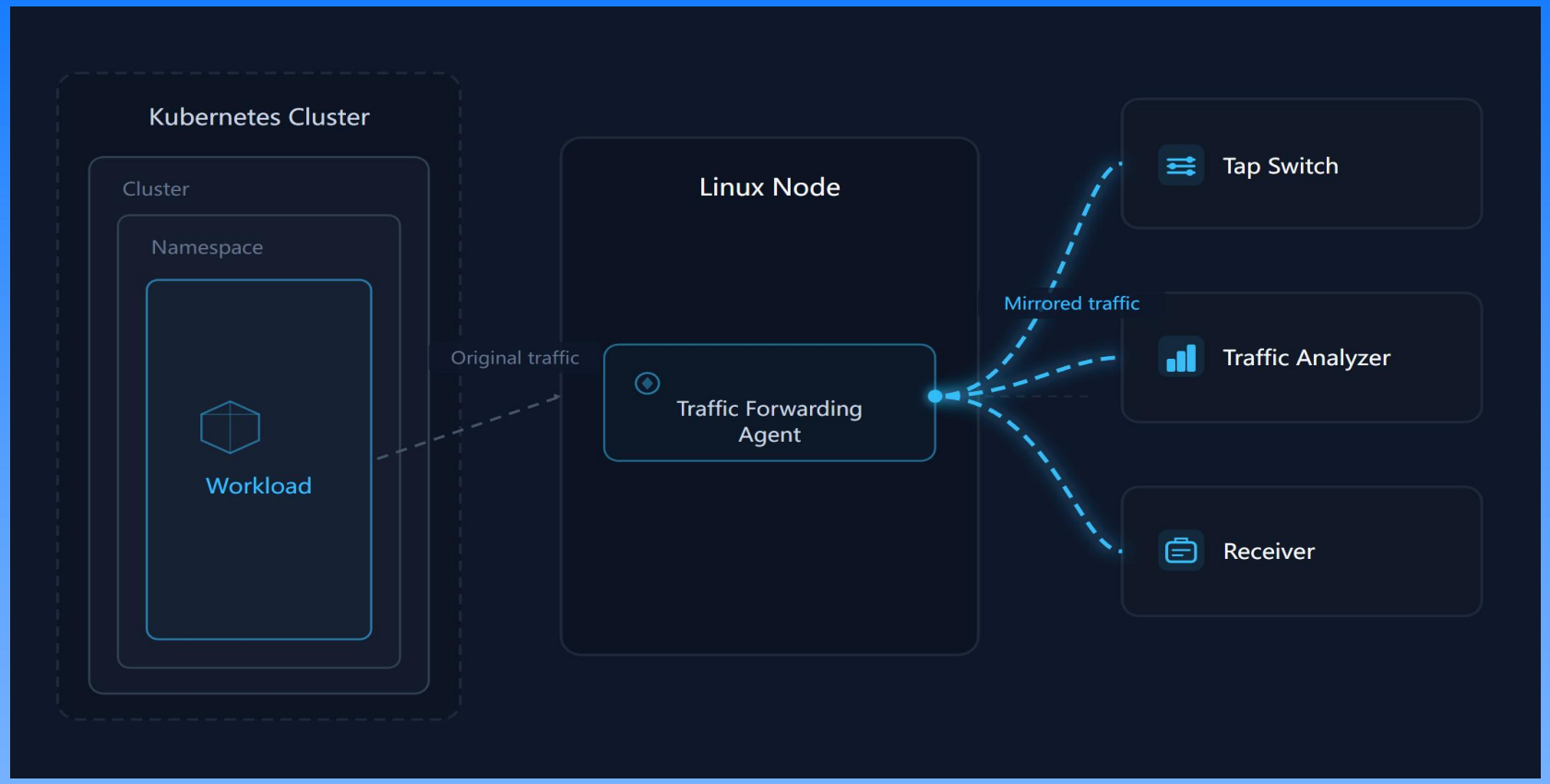


实践一：云原生流量转发，本质不是抓包而是按 K8s 语义选流量

- 按 K8s 语义选流量
  集群、Namespace、Workload、Pod IP / VIP

- 无侵入，不影响原流量
  原始业务路径保持不变

- 支持多接收端与无状态接收端
  包括 Tap 交换机 / 分析平台



业务诉求：要的不是全量抓包

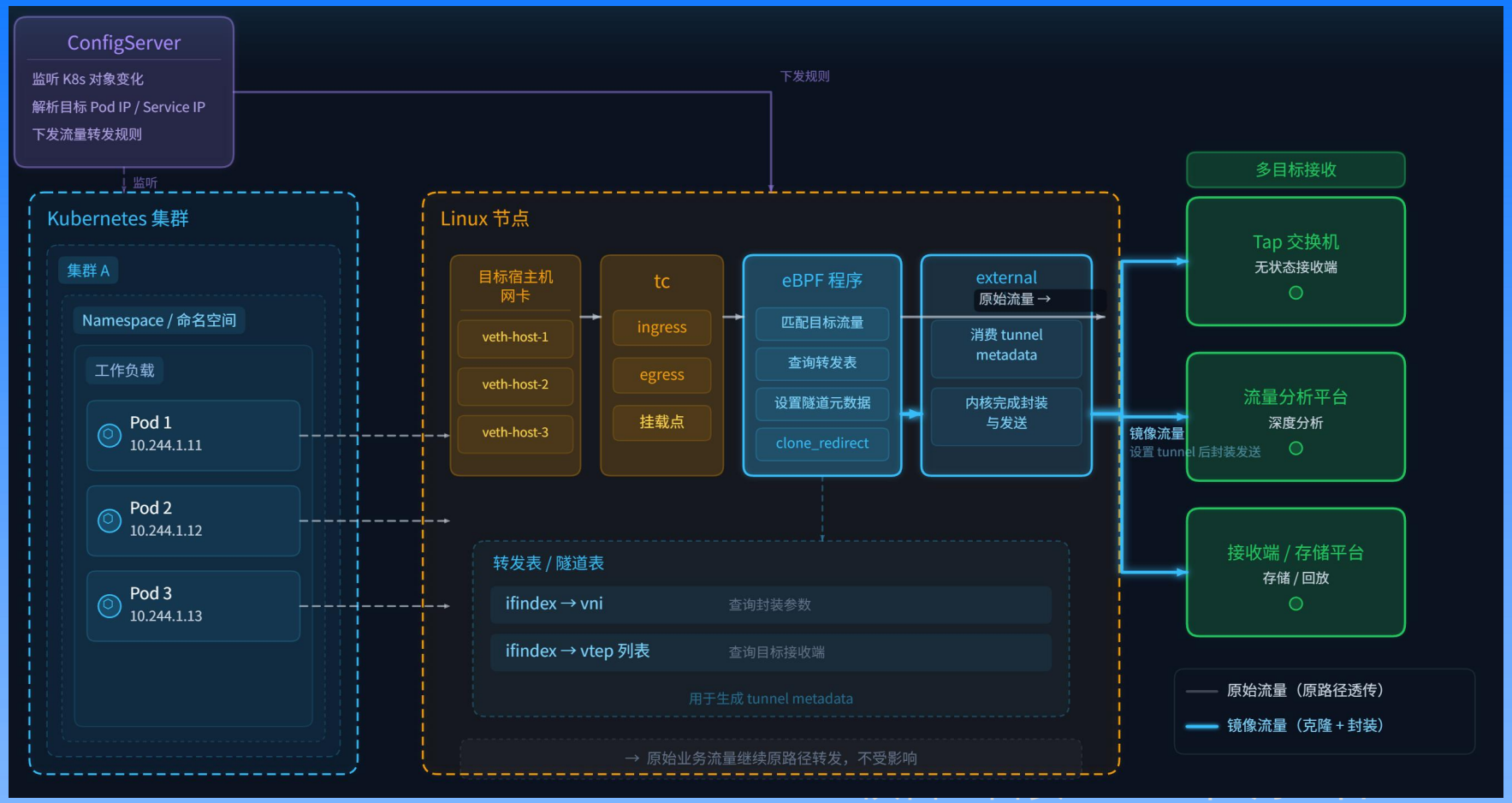
- 按 `Cluster / Namespace / Workload / Service IP / Pod IP` 选择目标流量
- 无侵入，不能影响原始业务流量
- 不依赖额外专用硬件
- 支持一个或多个接收端，包括无状态接收端

传统方案为何难以同时满足?

很难同时满足三个要求:
第一,按 K8s 语义选择流量;
第二,对原始业务流量无侵入;
第三,在不依赖额外硬件的前提下,低开销地转发到多个接收端。

总体方案：控制面+数据面

1. ConfigServer 监听 K8s 对象变化
2. 根据规则解析出目标 Pod IP / Service IP
3. 将目标规则下发到对应节点采集器
4. 采集器根据目标 IP 定位宿主机侧真实网卡
5. 在目标网卡挂载 tc ingress / egress ebpf
6. eBPF 查表后设置 tunnel metadata，再 clone_redirect 到 external vxlan，由内核完成封装和发送



为什么不在 eBPF 中手工封装 VXLAN

VXLAN vs External VXLAN

普通 `VXLAN / 非 external` 路径更容易走到自己封装包头、自己处理数据面的重路径；

`External VXLAN` 则是由 `eBPF` 写 `metadata`，再交给内核封装，所以更适合把云原生语义映射成转发信息。

VXLAN vs External VXLAN

差别在于 VNI 是预先绑定，还是按 tunnel metadata 动态指定

对比项	普通 VXLAN	EXTERNAL VXLAN
封装方式	需要在 eBPF 侧自己处理封装流程	eBPF 只写 tunnel metadata，由内核 vxlan 设备完成封装
包头处理	需要自己处理外层 L2/L3/UDP/VXLAN 头	不需要自己拼外层包头
数据面动作	往往需要 skb 扩缩容、补头、改包	只做 match、clone_redirect、metadata
对原流量影响	工程风险更高，更容易影响原包处理链路	原始流量继续原路径，方式更稳
VNI 绑定方式	更偏静态配置	通过 tunnel metadata 动态指定 VNI/VTEP
云原生语义映射	也能做，但多语义多 VNI 场景下管理更重	更适合把 Namespace、Workload、Pod IP 等语义映射成转发信息
管理复杂度	多 VNI 场景下更容易走向大量配置和设备管理	不需要频繁创建和管理大量 vxlan 设备
端口使用	一般沿用原有实现方式	会独占一个 VXLAN UDP 端口 独占端口
K8s 环境影响	一般沿用现有路径	可能和现有 4789 使用冲突，通常需要改用非 4789 端口 4789 冲突
接收端要求	取决于原有实现方式	需要对端或接收端配合解包 接收端解包

优势项 需注意限制

如何通过 IP 找到网卡，以及无状态 Tap 交换机如何收包

如何通过目标 IP 找到宿主机侧网卡

1. 通过 rtnetlink 获取宿主机侧网卡的 ifindex、peer_netnsid、peer_ifindex
2. 扫描 /proc/<pid>/ns/net，识别有效 netns，并获取对应 netnsid
3. 通过 setns() 进入目标 netns，拿到该命名空间下的 ifindex:ip 地址表
4. 用 peer_netnsid + peer_ifindex 反查 peer_ipv4，建立 target_ip → host_ifname 映射

目标宿主机侧网卡

无状态 Tap Switch 如何收包

1. 为接收端选定一个 underlay 目的 IP
2. 通过路由项指定该 IP 必须从目标物理口发出
3. 通过静态邻居项为该 IP 固定目的 MAC，避免 ARP 交互
4. 这样内核就会直接从指定链路发帧，Tap Switch 只需被动接收

Tap Switch / 接收端

实践二：面向指定 Pod 进程的 eBPF CPU / Memory Profiling

1.目标是对指定工作负载内进程做 CPU 和内存 profiling
2.节点侧由 agent-libs 负责 eBPF 采样、栈聚合和 map 取数
3.Kindling 负责符号化、组装标准 pprof、补充 K8s 标签
4.最终把 profile 推到火焰图展示后端
5.整体思路参考了bcc、pyroscope、parca等

实践问题一：三种开关/绑定方式为什么不同

三种追踪模式对比：触发模型 & 开销面

PERF_EVENT / SAMPLING
CPU 采样

CPU0 CPU1 CPU2 CPU3 CPU4 CPU5

周期性中断采样

↓

读取当前 PID

↓

PID map 过滤

↓

保留样本 ✓

时间驱动 | 按 CPU | 先采样后过滤

按时间采样

UPROBE / TARGET PID
uprobe + PID 绑定

目标进程地址空间 PID=1234

- malloc() probe
- calloc() probe
- realloc() probe

↓ 函数命中

分配事件记录

⚡ 仅在此进程内触发 — trap 面最小

命中函数才触发 | 仅目标 PID | trap 面小

命中函数触发

UPROBE / PID=-1, CPU≥0
uprobe + pid=-1, cpu≥0

共享库: libc.so → malloc() probe 注入

进程 A	mm0
进程 B	mm1
进程 C	mm2
进程 D	mm3

vma vma vma vma

probe 扩散到多个 mm/vma

⚠ 并非按 CPU 定时触发 — 是地址映射决定的

trap 面更大 | CPU context | 不是按 CPU 命中

仍由 mm + 地址决定命中

实践问题二：为什么 in-use memory 很难做多 PID profiling

allocation profiling 只需要对分配事件做栈聚合
in-use memory 需要同时跟踪分配和释放，维护对象生命周期状态
这意味着需要长期维护 ptr -> size/stack/metadata 这样的分配表
bcc memleak 也是“匹配 alloc/free + outstanding allocations + stack traces”这一路
它的公开文档明确提示：高频 alloc/free 场景会有显著开销，需要 sample rate 降低成本
所以内存不够的时候，单 PID 还能做，多 PID 同时做就容易把 map 和状态表撑大

实践问题三：为什么做不到对任意 C/Go 进程都稳定 profiling

为什么不能指望对任意进程都稳定做 Profiling

限制不只是能不能挂 probe，而是有没有稳定语义入口

C / C++

有条件可做

栈回溯需以下条件全部满足 ↓

- **帧指针** -fno-omit-frame-pointer ✓
- **符号表** 未 strip 的 binary ✓
- **调试信息** DWARF / .debug_info ✓
- **正确分配器入口** malloc / free 稳定符号 ✓

条件缺失 → 栈不可回溯 / 符号无法解析

LEFT
依赖符号与分配器
VS
RIGHT
依赖运行时语义

Go

天然更难

Go 进程 → runtime → ??? 不稳定

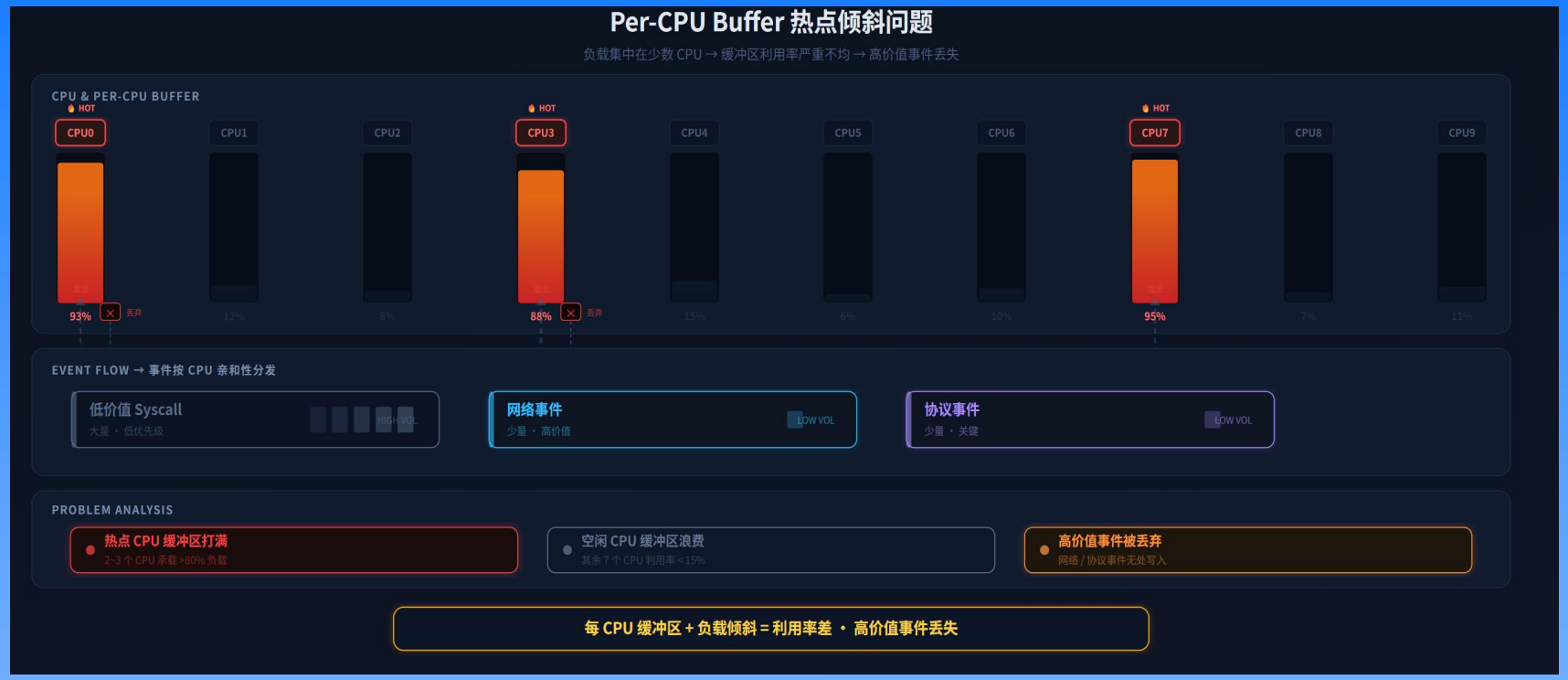
- `runtime.mallocgc` 分配入口被运行时包装 ✗
- GC 驱动释放 无显式 free 调用点 ✗
- 无稳定 free 释放由 GC sweep 触发 ✗
- 版本 / 内联变化 符号随版本和优化漂移 ✗

限制来自运行时语义本身，非配置可解

真正限制不是能不能挂 probe，而是有没有 稳定入口 · C/C++ 有条件可做 · Go 难点来自运行时

问题一：在有限探针资源下，如何尽量减少关键事件丢失

1、在真实环境里，探针可用的 CPU / 内存 / channel / 用户态消费线程 资源本来就是有限的
2、国产化场景里很多机器低于 5.8，无法使用BPF_MAP_TYPE_RINGBUF 更多只能依赖 BPF_MAP_TYPE_PERF_EVENT_ARRAY
BPF_MAP_TYPE_PERF_EVENT_ARRAY 本质上是 per-CPU perf buffer
3、在高核数机器上，buffer、fd、mmap 和轮询成本会随着 CPU 数一起上升
4、事件分布又往往集中在少数热点 CPU



BPF 事件过滤与处理方案

主链路 · 快速分流 · 丢弃路径

syscall / tracepoint / kprobe
进程 / 线程 / fd 事件 → 进入 ① 七层过滤主链路

① 事件类别过滤
只开必要 hook | events_mask

② 进程身份过滤
pid / tgid | comm

③ 容器 / 命名空间
cgroup | container | netns

④ fd → file → f_op
有 fd 吗 | fd → file | socket_file_ops

⑤ socket 语义与地址族
AF_INET / AF_INET6 | socket netns
---▶ AF_UNIX / 非目标网络族

⑥ L4 / 端口过滤
TCP / UDP | port 53 | 端口白名单
---▶ 非目标协议 / 端口

⑦ 应用层前探
HTTP | DNS | SQL | snaplen
---▶ 非目标应用协议

↓ 进入上下文语义 buffer

netdev / tc / skb
网卡收发包 · **只依赖五元组** · 直接进入五元组 buffer ↓

快速分流 ▼

上下文语义 buffer
承载进程 / 线程 / fd / socket 语义的事件
0 1 2 3 4 5 6 7
per-CPU ring buffer
热点 CPU → 更大 buffer

五元组快速分析 buffer
承载五元组 / 方向 / ifindex 网卡包事件
0 1 2 3 4 5 6 7
per-CPU ring buffer
热点 CPU → 更大 buffer

? 如何判断热点 CPU? 两种思路

1 经验法
在每个 NUMA 组内对前 N 个 CPU 做倾斜分配

2 实时水位法
根据占用率、丢包次数动态识别热点

问题二：中断返回路径插入其他采集点，复用状态区可能被覆盖

感谢倾听！

欢迎一起深入探讨 eBPF!

邮箱：254414795@qq.com